

Competitive Programming

Section 26 Editorial: Tree DP Basics

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	A. Festival Invite	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Algorithm	5
2.5	Correctness proof	5
2.6	Complexity	5
2.7	Full C++ solution	6
2.8	Full Python solution	7
2.9	Common wrong answers	7
3	B. Road Pairing	9
3.1	Restatement	9
3.2	Observations	9
3.3	Algorithm	9
3.4	Correctness proof	9
3.5	Complexity	9
3.6	Full C++ solution	9
3.7	Full Python solution	11
4	C. Tree Colorings	12
4.1	Restatement	12
4.2	Observations	12
4.3	Algorithm	12
4.4	Correctness proof	12
4.5	Complexity	12
4.6	Full C++ solution	12
4.7	Full Python solution	13
5	D. Road Guards	15
5.1	Restatement	15
5.2	Constraints and traps	15
5.3	State and walk-through	15
5.4	Algorithm	16
5.5	Correctness proof	16
5.6	Complexity	17
5.7	C++ implementation notes	17
5.8	Python implementation notes	17
5.9	Full C++ solution	17
5.10	Full Python solution	18
5.11	Common wrong answers	19
5.12	Tests that matter	19

1 Editorial overview

Problem	State	Combination
A. Festival Invite	choose or skip node	sum child optima
B. Road Pairing	match node to zero/one child	best child upgrade
C. Tree Colorings	color of node	product over children
D. Road Guards	activate or leave node inactive	force active children when needed

All solutions root the tree at vertex 1 and process nodes in postorder.

2 A. Festival Invite

2.1 Restatement

Choose a set of vertices with maximum total happiness such that no chosen vertices are adjacent.

2.2 Constraints and traps

n is large, so brute force over subsets is impossible. A chain can be long, so recursive DFS may overflow in Python or C++ environments.

2.3 Observations

If u is invited, none of its children may be invited. If u is not invited, each child subtree can choose independently whether to invite the child.

2.4 Algorithm

```
root the tree
for u in postorder:
    take[u] = h[u] + sum(skip[v])
    skip[u] = sum(max(take[v], skip[v]))
print max(take[root], skip[root])
```

2.5 Correctness proof

Theorem. $\text{take}[u]$ and $\text{skip}[u]$ have the stated optimal meanings.

Proof. Induct on subtree height. Leaves are immediate. For an internal node u , if u is taken, every child must be skipped, and child subtrees are independent. If u is skipped, every child can use its better state. By the induction hypothesis, the child values are optimal, so the formulas are optimal. $\square$

2.6 Complexity

Time: $O(n)$
Memory: $O(n)$

2.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<long long> h(n);
    for (long long &x : h) cin >> x;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> parent(n, -1), order;
    order.push_back(0);
    parent[0] = 0;
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (parent[v] == -1) {
            parent[v] = u;
            order.push_back(v);
        }
    }
    vector<long long> take = h, skip(n, 0);
    for (int idx = n - 1; idx >= 0; idx--) {
        int u = order[idx];
        for (int v : adj[u]) if (parent[v] == u) {
            take[u] += skip[v];
            skip[u] += max(take[v], skip[v]);
        }
    }
    cout << max(take[0], skip[0]) << '\n';
    return 0;
}
```

2.8 Full Python solution

```
import sys

def postorder(adj: list[list[int]], root: int = 0) -> tuple[list[int], list[int]]:
    n = len(adj)
    parent = [-1] * n
    order = [root]
    parent[root] = root
    for u in order:
        for v in adj[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)
    return parent, order[::-1]

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    h = [next(it) for _ in range(n)]
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)
    parent, order = postorder(adj)
    take = h[:]
    skip = [0] * n
    for u in order:
        for v in adj[u]:
            if parent[v] == u:
                take[u] += skip[v]
                skip[u] += max(take[v], skip[v])
    print(max(take[0], skip[0]))

if __name__ == "__main__":
    main()
```

2.9 Common wrong answers

Taking a node and also allowing chosen children.
Processing nodes in BFS order instead of postorder.
Using int for large happiness sums.

3 B. Road Pairing

3.1 Restatement

Find the largest set of edges in a tree such that no two chosen edges share a vertex.

3.2 Observations

For a node u , first assume it is not matched to a child. Then every child can use its best state. To match u to one child v , replace v 's best state by $\text{free}[v]$ and add one chosen edge.

3.3 Algorithm

```
base = sum(max(free[v], used[v])) over children
free[u] = base
used[u] = max(base, base - max(free[v], used[v]) + free[v] + 1)
```

3.4 Correctness proof

Theorem. The transition computes the maximum matching in every subtree.

Proof. Induct on subtree height. If u is not matched to a child, children are independent and contribute their best values. If u is matched to a child v , no other child edge incident to u can be chosen, and v cannot be matched inside its child subtrees; hence $\text{free}[v] + 1$ replaces v 's previous best contribution. Trying every child covers every possible edge incident to u . $\square$

3.5 Complexity

```
Time:  $O(n)$ 
Memory:  $O(n)$ 
```

3.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> parent(n, -1), order;
    parent[0] = 0;
    order.push_back(0);
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (parent[v] == -1) {
            parent[v] = u;
            order.push_back(v);
        }
    }
    vector<int> free(n, 0), used(n, 0);
    for (int idx = n - 1; idx >= 0; idx--) {
        int u = order[idx];
        int base = 0;
        vector<int> children;
        for (int v : adj[u]) if (parent[v] == u) {
            children.push_back(v);
            base += max(free[v], used[v]);
        }
        free[u] = base;
        int best = base;
        for (int v : children) {
            best = max(best, base - max(free[v], used[v]) + free[v] + 1);
        }
        used[u] = best;
    }
    cout << max(free[0], used[0]) << '\n';
    return 0;
}

```

3.7 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)
    parent = [-1] * n
    parent[0] = 0
    order = [0]
    for u in order:
        for v in adj[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)
    free = [0] * n
    used = [0] * n
    for u in reversed(order):
        base = 0
        children = []
        for v in adj[u]:
            if parent[v] == u:
                children.append(v)
                base += max(free[v], used[v])
        free[u] = base
        best = base
        for v in children:
            cand = base - max(free[v], used[v]) + free[v] + 1
            if cand > best:
                best = cand
        used[u] = best
    print(max(free[0], used[0]))

if __name__ == "__main__":
    main()
```

4 C. Tree Colorings

4.1 Restatement

Color every vertex white or black so that no edge has two black endpoints. Count valid colorings modulo 1_000_000_007.

4.2 Observations

If u is white, each child may be white or black. If u is black, every child must be white.

4.3 Algorithm

```
white[u] = product(white[v] + black[v])
black[u] = product(white[v])
answer = white[root] + black[root]
```

4.4 Correctness proof

Theorem. The DP counts exactly all valid colorings of each rooted subtree.

Proof. Induct on subtree height. A leaf has one white and one black coloring. For an internal node, fixing the color of u makes child subtrees independent. White u imposes no black constraint on children, while black u forces every child root white. Multiplying counts independent choices across children. $\square$

4.5 Complexity

```
Time:  $O(n)$ 
Memory:  $O(n)$ 
```

4.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    const long long MOD = 1000000007LL;
    int n;
    if (!(cin >> n)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> parent(n, -1), order;
    parent[0] = 0;
    order.push_back(0);
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (parent[v] == -1) {
            parent[v] = u;
            order.push_back(v);
        }
    }
    vector<long long> white(n, 1), black(n, 1);
    for (int idx = n - 1; idx >= 0; idx--) {
        int u = order[idx];
        for (int v : adj[u]) if (parent[v] == u) {
            white[u] = white[u] * ((white[v] + black[v]) % MOD) % MOD;
            black[u] = black[u] * white[v] % MOD;
        }
    }
    cout << (white[0] + black[0]) % MOD << '\n';
    return 0;
}

```

4.7 Full Python solution

```

import sys

MOD = 1_000_000_007

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)
    parent = [-1] * n
    parent[0] = 0
    order = [0]
    for u in order:
        for v in adj[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)
    white = [1] * n
    black = [1] * n
    for u in reversed(order):
        for v in adj[u]:
            if parent[v] == u:
                white[u] = white[u] * (white[v] + black[v]) % MOD
                black[u] = black[u] * white[v] % MOD
    print((white[0] + black[0]) % MOD)

if __name__ == "__main__":
    main()

```

5 D. Road Guards

5.1 Restatement

Every tree vertex has a positive activation cost. Activate some vertices so that every tree edge touches at least one activated endpoint, while minimizing the sum of activated costs. A one-vertex tree has no roads to cover, so its optimal cost is zero.

5.2 Constraints and traps

n can be 200000 and costs can reach 10^9 . Enumerating activated subsets is exponential, and a recursive traversal can fail on a path-shaped tree.

An activated vertex covers every incident edge, not only edges to children.
If a parent is inactive, every child must be active to cover the parent-child road.
If a parent is active, each child may independently be active or inactive.
The answer may leave the root inactive.

Use `long long` in C++ for the total cost.

5.3 State and walk-through

Root the tree at vertex 1. Define two subtree costs:

```
active[u]    = minimum cost in u's subtree when u is activated
inactive[u]  = minimum cost in u's subtree when u is not activated
```

If u is active, edge (u, v) is already covered for every child v , so that child can use its cheaper state. If u is inactive, each child must be active to cover edge (u, v) :

```
active[u]    = cost[u] + sum(min(active[v], inactive[v]))
inactive[u]  =          sum(active[v])
```

For the sample, process leaves first:

```
vertex 2: active=2, inactive=0
vertex 4: active=1, inactive=0
vertex 5: active=3, inactive=0

vertex 3: active    = 4 + min(1,0) + min(3,0) = 4
          inactive  = 1 + 3 = 4
```

```
vertex 1: active    = 5 + min(2,0) + min(4,4) = 9
          inactive = 2 + 4 = 6
```

The root has no parent edge that needs coverage, so either state is legal. The answer is $\min(9, 6) = 6$, obtained by activating vertices 2 and 3.

5.4 Algorithm

```
root the tree at vertex 1 and record an iterative traversal order
active[u] = cost[u], inactive[u] = 0 for every vertex

for u in reverse traversal order:
    for every child v of u:
        active[u] += min(active[v], inactive[v])
        inactive[u] += active[v]

print min(active[root], inactive[root])
```

Reverse traversal order is postorder: all child state pairs are ready before their parent uses them.

5.5 Correctness proof

Lemma 1. For every vertex u , $\text{active}[u]$ is the minimum cost of a valid activation choice inside u 's subtree under the condition that u is active.

Proof. Induct on subtree height. For a leaf, activating it costs exactly $\text{cost}[u]$. For an internal u , every edge from u to a child is covered by the active endpoint u . Thus each child subtree may choose either valid child state independently. By the induction hypothesis, the least contribution of child v is $\min(\text{active}[v], \text{inactive}[v])$; adding the mandatory $\text{cost}[u]$ gives exactly the transition. $\square$

Lemma 2. For every vertex u , $\text{inactive}[u]$ is the minimum cost of a valid activation choice inside u 's subtree under the condition that u is inactive.

Proof. A leaf needs no activated vertex in its subtree, so its cost is zero. For an internal inactive u , each edge (u,v) must be covered by its child endpoint, which forces every child v active. Child subtrees are independent, and by Lemma 1 their least valid contributions are $\text{active}[v]$. Their sum is precisely the transition for $\text{inactive}[u]$. $\square$

Theorem. The algorithm prints the minimum activation cost covering every road in the tree.

Proof. Lemmas 1 and 2 prove the meanings of both root states. The root has no parent edge, so it may be active or inactive. Taking the smaller of the two valid root costs is therefore the global optimum. $\square$

5.6 Complexity

Each vertex and edge is processed a constant number of times.

```
Time:    O(n)
Memory:  O(n)
```

5.7 C++ implementation notes

Build `parent` and `order` iteratively, then scan `order` backward. Initialize `active` with vertex costs and `inactive` with zero; this already gives the correct leaf base case. Use `long long` for both arrays.

5.8 Python implementation notes

The pattern for `u in order:` is safe even while appending newly discovered children to the list. Then `reversed(order)` provides postorder without recursion. Python integers safely hold every possible cost total.

5.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> cost(n);
    for (long long &value : cost) cin >> value;
    vector<vector<int>> graph(n);
    for (int i = 0; i + 1 < n; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> parent(n, -1), order = {0};
    parent[0] = 0;
    for (int index = 0; index < static_cast<int>(order.size()); ++index) {
        int u = order[index];
```

```

        for (int v : graph[u]) {
            if (parent[v] == -1) {
                parent[v] = u;
                order.push_back(v);
            }
        }
    }

    vector<long long> active = cost, inactive(n, 0);
    for (int index = n - 1; index >= 0; --index) {
        int u = order[index];
        for (int v : graph[u]) {
            if (parent[v] == u) {
                active[u] += min(active[v], inactive[v]);
                inactive[u] += active[v];
            }
        }
    }

    cout << min(active[0], inactive[0]) << '\n';
}

```

5.10 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    cost = data[1:1 + n]
    graph = [[] for _ in range(n)]
    index = 1 + n
    for _ in range(n - 1):
        u = data[index] - 1
        v = data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    parent = [-1] * n
    parent[0] = 0
    order = [0]
    for u in order:
        for v in graph[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)

```

```

active = cost[:]
inactive = [0] * n
for u in reversed(order):
    for v in graph[u]:
        if parent[v] == u:
            active[u] += min(active[v], inactive[v])
            inactive[u] += active[v]

print(min(active[0], inactive[0]))

if __name__ == "__main__":
    main()

```

5.11 Common wrong answers

- Applying the independent-set recurrence, which reverses the required child state when the parent is active/inactive.
- Forcing the root active even though it has no parent road to cover.
- Adding `min(active[v], inactive[v])` to `inactive[u]`; an inactive parent forces an active child.
- Recursing into a long path and overflowing the call stack.
- Using a 32-bit integer for costs.

5.12 Tests that matter

- a one-vertex tree, whose answer is zero;
- a long path, which checks iterative traversal and alternating choices;
- a star with an expensive center, which can favor activating all leaves;
- a star with cheap center, which favors the center;
- equal costs, where several optimal guard sets exist;
- costs near 10^9 on a large tree, which require 64-bit totals.

6 Testing and carry-forward

The section checker runs:

```
CP_TARGET=solution python3 sections/26_tree_dp_basics/check.py
```

Tests include single-node trees, paths, stars, and random trees. For Road Guards, the local generator enumerates every activated subset on trees up to 18 vertices, then uses a 200000-vertex path as the scale case.

Carry forward:

```
Root the tree.  
Define states around the node's relationship to its parent/children.  
Process children before parents.  
Combine child subtrees independently.
```